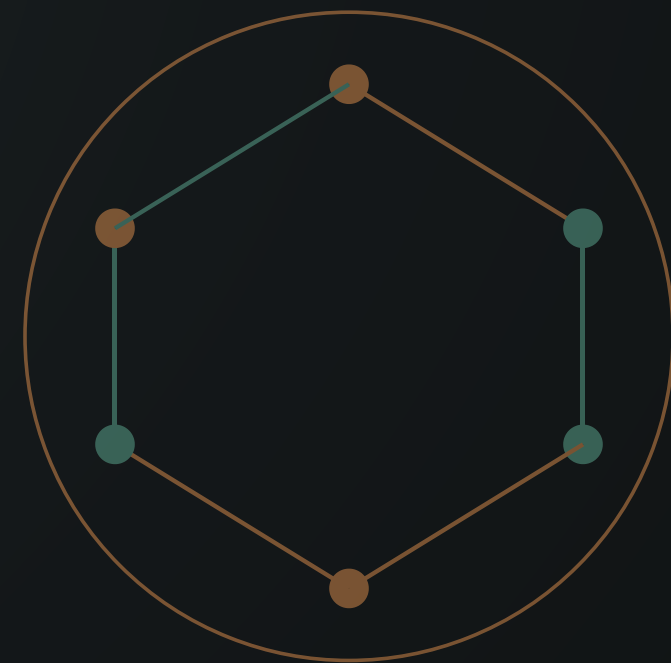


— NỀN TẢNG TRIỂN KHAI HỆ THỐNG

Docker: từ máy cá nhân tới server thật

DOCKER • COMPOSE • VOLUME • TRIỂN KHAI LÊN VPS



Câu nói kinh điển của mọi lập trình viên

"Ủa lạ, máy tui chạy được mà."

Code chạy ngon trên máy bạn. Gửi cho bạn cùng nhóm thì lỗi. Đưa lên server thì lỗi kiểu khác. Đêm trước ngày demo, cả đội ngồi cài lại từ đầu.

Docker sinh ra để chấm dứt câu nói này. Nguyên tắc: thay vì gửi code rồi dặn người khác cài giống mình, bạn gửi luôn cả cái máy đã cài sẵn mọi thứ.

Vì sao cùng một code lại chạy khác nhau

Máy của bạn

Windows 11

Node 22

PostgreSQL 17

Chạy ngon

Máy bạn cùng nhóm

macOS

Node 18

Chưa cài PostgreSQL

Lỗi ngay dòng đầu

Server thuê

Ubuntu Server

Node 16 cũ mèm

Thiếu thư viện hệ thống

Lỗi kiểu khác nữa

Code giống nhau, nhưng môi trường xung quanh code thì khác nhau

Docker giải quyết bằng cách đóng gói luôn cả môi trường đó đi cùng code

Docker là gì

Tên Docker lấy từ container hàng hoá. Trước khi có container, mỗi loại hàng bốc dỡ một kiểu, mỗi cảng một chuẩn. Từ khi có thùng container tiêu chuẩn, mọi cảng và mọi con tàu trên thế giới đều xử lý được, không cần quan tâm bên trong là gạo hay tivi.

Không có Docker

Muốn chạy sản phẩm ở đâu thì phải cài lại từng thứ ở đó: đúng phiên bản ngôn ngữ, đúng thư viện, đúng cấu hình, đúng thứ tự. Sai một chỗ là hỏng.

Mỗi máy mới là một lần cài lại từ đầu, và lần nào cũng gặp lỗi khác nhau.

Có Docker

Bạn đóng gói code cùng toàn bộ môi trường của nó vào một thùng tiêu chuẩn. Thùng đó chạy giống hệt nhau trên máy bạn, máy bạn cùng nhóm, và trên server.

Máy nào có Docker là chạy được, không cần cài gì thêm.

Ba từ phải nhớ: Dockerfile, Image, Container



Ví như nấu ăn: công thức là Dockerfile, món đã nấu đóng hộp là Image, hộp đang mở ra ăn là Container

Một Image có thể tạo ra 10 Container giống hệt nhau, giống như một hộp nhân ra nhiều bản

Container khác máy ảo ở chỗ nào

MÁY ẢO

Mỗi máy ảo công nguyên một hệ điều hành riêng

Muốn chạy 3 ứng dụng thì có 3 hệ điều hành cùng chạy. Tốn vài GB bộ nhớ mỗi cái, khởi động mất vài phút.

File máy ảo thường nặng hàng chục GB.

CONTAINER

Các container dùng chung nhân hệ điều hành của máy chủ

Chỉ đóng gói phần cần thiết cho ứng dụng. Khởi động trong vài giây, tốn vài chục tới vài trăm MB.

Vì nhẹ nên một server nhỏ chạy được cả chục container.

Hệ quả thực tế: với VPS rẻ tiền mà đội thi hay dùng, chạy máy ảo gần như không khả thi, còn Docker thì thoải mái chạy web, cơ sở dữ liệu và bộ nhớ đệm cùng lúc.

Một Dockerfile thật trông như thế nào

DOCKERFILE

```
FROM node:22-alpine
WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

EXPOSE 3000
CMD ["node", "server.js"]
```

FROM - bắt đầu từ đâu

Lấy sẵn một hộp đã cài Node phiên bản 22. Chữ alpine nghĩa là bản rút gọn, nhẹ hơn nhiều lần

WORKDIR - làm việc ở thư mục nào

Mọi lệnh sau đó chạy trong thư mục /app bên trong hộp

COPY package rồi RUN npm trước

Đây là mẹo quan trọng: chép danh sách thư viện và cài trước, chép code sau. Nhờ vậy sửa code không phải cài lại thư viện, build nhanh hơn nhiều lần

EXPOSE và CMD

Báo hộp này mở cổng 3000, và khi chạy thì thực thi lệnh nào

Sản phẩm thật không chỉ có một phần

Một sản phẩm DX-OS đơn giản đã cần ít nhất ba thứ chạy song song



Bật tay từng cái, nhớ đúng thứ tự, nhớ đúng tham số. Rất dễ sai.

Docker Compose gom cả ba vào một file, bật bằng một lệnh duy nhất.

Cả dàn máy gói trong một file

COMPOSE.YAML

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://app:matkhou@db:5432/mydb
    depends_on:
      - db

  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: matkhou
      POSTGRES_DB: mydb
    volumes:
      - dbdata:/var/lib/postgresql/data

volumes:
  dbdata:
```

Gọi nhau bằng tên

Trong địa chỉ kết nối có chữ **@db**. Đó chính là tên dịch vụ khai báo bên dưới. Các container tự tìm thấy nhau bằng tên, không cần biết địa chỉ IP

Dòng volumes cứu dữ liệu của bạn

Không có dòng này thì tắt container là mất sạch dữ liệu trong cơ sở dữ liệu

Không cần dòng version nữa

Nhiều hướng dẫn cũ bắt ghi **version: "3.8"** ở đầu file. Bản Compose hiện nay bỏ qua dòng đó, viết vào chỉ thừa

Những lệnh dùng hàng ngày

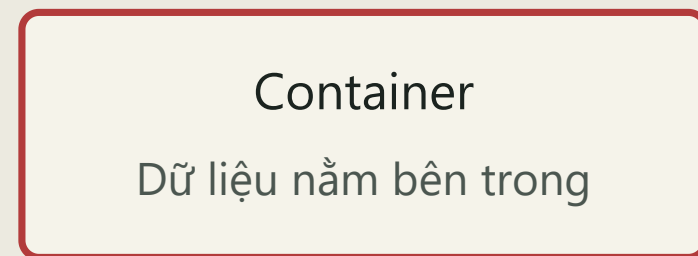
LỆNH	LÀM GÌ
<code>docker compose up -d</code>	Bật toàn bộ dàn máy, chạy ngầm phía sau
<code>docker compose ps</code>	Xem cái gì đang chạy, cái nào chết
<code>docker compose logs -f web</code>	Xem log của dịch vụ web, chạy tới đâu hiện tới đó. Đây là chỗ đầu tiên cần nhìn khi có lỗi
<code>docker compose up -d --build</code>	Sửa code xong thì đóng gói lại rồi bật lại
<code>docker compose exec db psql -U app</code>	Chui vào bên trong container để thao tác trực tiếp
<code>docker compose down</code>	Tắt và dọn dẹp, nhưng dữ liệu trong volume vẫn còn
<code>docker compose down -v</code>	Cẩn thận: chữ v phía sau xoá luôn volume, nghĩa là mất sạch dữ liệu. Nhiều người gõ nhầm và mất hết bài làm

Lưu ý về cú pháp: lệnh đúng hiện nay là **docker compose** viết cách nhau. Bản cũ **docker-compose** có dấu gạch nối đã hết vòng đời từ tháng 5/2024, tài liệu nào còn dạy kiểu đó là tài liệu cũ.

Volume: nơi dữ liệu sống sót khi container chết

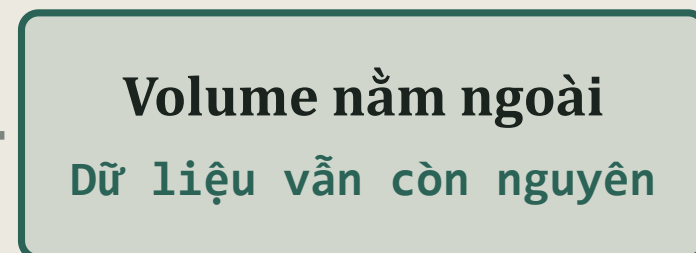
Đây là chỗ người mới mất dữ liệu nhiều nhất. Hiểu sai một lần là mất toàn bộ dữ liệu thử nghiệm.

KHÔNG DÙNG VOLUME



Toàn bộ đơn hàng, tài khoản, ảnh đã tải lên

CÓ DÙNG VOLUME



Bật container mới lên là dùng lại được ngay

Quy tắc dễ nhớ: container là thứ dùng xong vứt đi được. Cái gì mất đi mà bạn tiếc thì phải nằm trong volume, không được nằm trong container.

Các container nói chuyện với nhau thế nào

Bên trong: gọi nhau bằng tên

Compose tự tạo một mạng riêng cho cả nhóm. Container web muốn gọi cơ sở dữ liệu thì chỉ cần ghi tên **db**, không cần biết địa chỉ IP.

Tên đó chính là tên bạn đặt trong phần services của file compose.

Ra ngoài: phải mở cổng

Dòng **ports: "3000:3000"** nghĩa là mở cổng 3000 của máy thật, nối vào cổng 3000 bên trong container.

Không có dòng này thì container vẫn chạy, nhưng trình duyệt không vào được.

Lỗi kinh điển của người mới: trong code ghi địa chỉ cơ sở dữ liệu là **localhost**. Bên trong container, chữ localhost nghĩa là chính container đó, không phải máy thật. Phải ghi tên dịch vụ, tức là **db**.

Mật khẩu và khoá bí mật để ở đâu

CÁCH LÀM NGUY HIỂM

Viết thẳng mật khẩu vào code rồi đẩy lên GitHub

Repo của cuộc thi là repo công khai. Mật khẩu cơ sở dữ liệu hay khoá dịch vụ AI đẩy lên đó là ai cũng đọc được.

Xoá đi rồi commit lại cũng vô ích, vì lịch sử Git vẫn giữ nguyên bản cũ.

CÁCH LÀM ĐÚNG

Đặt trong file .env và thêm file đó vào .gitignore

Code chỉ đọc giá trị từ biến môi trường, không chứa giá trị thật.

Kèm thêm một file mẫu **.env.example** ghi tên các biến nhưng để trống giá trị, để người khác biết cần điền gì.

Thể lệ cuộc thi yêu cầu **công bố rõ thành phần và dịch vụ bên thứ ba đã dùng**, nhưng công bố tên dịch vụ hoàn toàn khác với để lộ khoá truy cập. File .env.example là cách làm đúng cả hai việc.

Đừng gói cả rác vào hộp

.DOCKERIGNORE

```
# thư viện, để Docker tự cài lại
node_modules

# lịch sử Git, không cần trong hộp
.git

# bí mật, tuyệt đối không đưa vào
.env

# file tạm và log
*.log
tmp/

# tài liệu, không ảnh hưởng lúc chạy
README.md
docs/
```

Không có file này thì sao

Lệnh COPY sẽ chép cả thư mục node_modules từ máy bạn vào hộp. Hộp phình từ 100 MB lên vài GB, build lâu gấp nhiều lần

Nguy hiểm hơn

File .env chứa mật khẩu cũng bị chép vào hộp. Ai lấy được hộp đó là đọc được mật khẩu của bạn

Cách nghĩ đúng

Chỉ đưa vào hộp đúng những gì cần để chạy. Mọi thứ khác đều là gánh nặng hoặc rủi ro

Từ máy cá nhân lên server thật

- 1 **Thuê một VPS** và cài Docker lên đó. Đây là máy tính chạy 24/7 ở trung tâm dữ liệu, có địa chỉ công khai để ai cũng truy cập được
- 2 **Đẩy code lên GitHub**, rồi từ server tải code về. Không chép file bằng tay, vì lần sau cập nhật sẽ không biết đã đổi gì
- 3 **Tạo file .env trên server** với mật khẩu thật, khác mật khẩu lúc chạy thử ở nhà
- 4 **Chạy docker compose up -d** và kiểm tra bằng docker compose ps xem có gì chết không
- 5 **Trỏ tên miền về server** và bật chứng chỉ bảo mật để địa chỉ có https, không hiện cảnh báo đỏ

Nhà tài trợ AZDIGI tặng 25 hosting thời hạn 3 tháng cho các đội thi. Đây đúng là thứ cần cho bước này, các đội nên tận dụng để có sản phẩm chạy thật trên mạng thay vì chỉ chạy trên laptop.

Hộp mang lên server phải nhẹ

Dụng cụ để nấu ăn không cần mang theo khi giao món. Cách làm hai chặng: chặng đầu dùng đủ công cụ để đóng gói, chặng sau chỉ lấy thành phẩm.

DOCKERFILE HAI CHẶNG

```
# Chặng 1: có đủ công cụ để build
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Chặng 2: chỉ lấy thành phẩm
FROM node:22-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

Vì sao phải nhẹ

Hộp nhẹ thì tải lên server nhanh, khởi động nhanh, tốn ít dung lượng. Hộp 1 GB và hộp 150 MB chênh nhau rất rõ khi bạn phải cập nhật nhiều lần trong ngày demo

Ghim đúng phiên bản

Ghi **node:22-alpine** chứ đừng ghi **node:latest**. Chữ latest nghĩa là hôm nay build ra một kiểu, tháng sau build ra kiểu khác, và bạn không biết vì sao hỏng

Chọn bản alpine

Bản rút gọn, chỉ chứa thứ cần thiết. Nhẹ hơn bản đầy đủ nhiều lần

Đặt một lớp đứng trước ứng dụng



Chỉ Nginx mở cổng ra ngoài. Các container còn lại nằm trong mạng riêng, an toàn hơn hẳn.

Sáu lỗi người mới hay mắc nhất

LỖI	HẬU QUẢ	CÁCH TRÁNH
Quên khai báo volume	Mất sạch dữ liệu khi tắt container	Mọi cơ sở dữ liệu đều phải có volume
Gõ nhầm down -v	Xoá luôn volume, mất hết bài làm	Nhớ kỹ: chữ v là xoá dữ liệu
Đẩy file .env lên Git	Lộ mật khẩu, lộ khoá dịch vụ AI	Thêm .env vào .gitignore ngay từ đầu
Dùng tag latest	Hôm nay chạy, mai build lại thì hỏng	Ghi rõ số phiên bản
Ghi localhost trong container	Không kết nối được cơ sở dữ liệu	Dùng tên dịch vụ trong compose
Không có .dockerignore	Hộp phình to, build chậm, lộ bí mật	Tạo file này ngay khi tạo Dockerfile

Việc cần làm cho sản phẩm dự thi

- 1 Viết **Dockerfile** cho sản phẩm, ghim rõ số phiên bản, dùng bản alpine cho nhẹ
- 2 Tạo **.dockerignore** ngay cùng lúc, đừng để sau mới nhớ ra
- 3 Viết **compose.yaml** gom web, cơ sở dữ liệu và các phần khác, nhớ khai báo volume cho dữ liệu
- 4 Chuyển mọi mật khẩu sang **.env**, thêm vào .gitignore, kèm một file .env.example để người khác biết cần điền gì
- 5 **Thử trên máy sạch:** nhờ bạn cùng nhóm tải repo về và chạy đúng một lệnh xem có lên không
- 6 Ghi vào **README** đúng các lệnh cần chạy. Giám khảo chấm mục "biên dịch được từ mã nguồn", họ sẽ làm y hệt

Docker mở đường cho những thứ đã học

Nhân bản

Thêm máy khi đông người

Vì container giống hệt nhau, muốn tăng sức chịu tải chỉ cần chạy thêm vài bản và đặt người gác cổng chia việc

Sự cố

Container chết thì tự bật lại

Khai báo một dòng restart trong compose là Docker tự dựng lại khi có gì chết, không cần người trực đêm

Sao lưu

Volume là thứ cần sao lưu

Vì mọi dữ liệu quan trọng nằm trong volume, việc sao lưu trở nên rõ ràng: sao lưu volume, không phải sao lưu container

Thứ tự học đúng là: **đóng gói chạy được trước, rồi mới tính chuyện chịu tải**. Một sản phẩm chưa deploy được thì bàn chuyện 100.000 người là quá sớm.